Ensembler: Protect Collaborative Inference Privacy from Model Inversion Attack via Selective Ensemble

Dancheng Liu, Chenhui Xu, Jiajie Li, Amir Nassereldine, Jinjun Xiong
Department of Computer Science and Engineering
University at Buffalo, NY, USA
{dliu37, cxu26, jli433, amirnass, jinjun}@buffalo.edu

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132673

***Abstract*—For collaborative inference through a cloud computing platform, it is sometimes essential for the client to shield its sensitive information from the cloud provider. In this paper, we introduce Ensembler, an extensible framework designed to substantially increase the difficulty of conducting model inversion attacks by adversarial parties. Ensembler leverages selective model ensemble on the adversarial server to obfuscate the reconstruction of the client's private information. Our experiments demonstrate that Ensembler can effectively shield input images from reconstruction attacks, even when the client only retains one layer of the network locally. Ensembler significantly outperforms baseline methods by up to 43.5% in structural similarity while only incurring 4.8% time overhead during inference.**

I. Introduction

Deep learning models have achieved exceptional performance across critical domains [1], but their increasing size poses challenges for computationally constrained edge devices like mobile phones [2, 3]. When the model's size and complexity exceed the computational ability of the edge devices, a prevalent solution to reduce the workload on these resource-limited edge devices is through collaborative inference (CI) [4], as shown in the left side of Fig. 1. During inference, the client (edge device) computes the first few layers of the network and sends the intermediate features to the server (cloud), where the server computes the computationally expensive layers and returns the features or the results back to the client. However, this raises concerns about the privacy of sensitive client data, especially when the cloud server may be adversarial and tasks involve sensitive information like medical or facial data [5]. Consequently, there is a growing need for secure, accurate, and efficient machine learning frameworks that enable privacy-preserving collaboration between edge devices and cloud servers.

During CI, a classical yet powerful attack on the client's sensitive data is through model inversion attacks (MIA). As shown in the right side of Fig. 1, an adversarial server aims to inverse the client's private network to reconstruct the private input through intermediate features. We will elaborate on the details of MIA in Section II. He et. al. show that the client's data is especially vulnerable when the client retains only one

The research was supported in part by the National AI Institute for Exceptional Education (NSF Award #2229873), Center for Early Literacy and Responsible AI (IES Award #R305C240046), FuSe-TG (NSF Award #2235364) and SaTC (NSF Award #2329704). The opinions expressed are those of the authors and do not represent views of any sponsors.

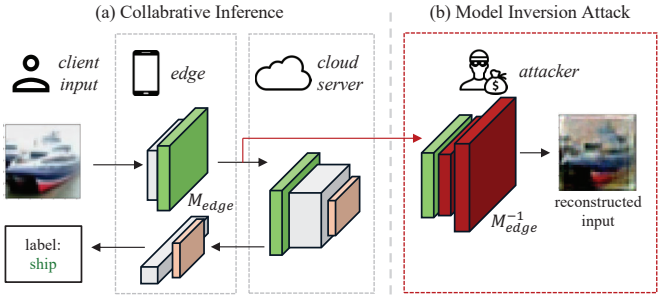

Fig. 1. An illustration of (a) collaborative inference, where the client offloads computation to the server; and (b) model inversion attack, where the adversarial server builds a decoder M_{edge}^{-1} to decode the client's input from the intermediate features computed by the client's private network M_{edge}.

single convolutional layer [4]. While the client can potentially keep more privacy as it keeps more layers, such a method is less practical in the real world due to the limitations of the computational power of edge devices. Shredder [4] uses a noise injection layer before the client sends out computed results to reduce mutual information between client and server while maintaining good classification accuracy. Nevertheless, Preva [5] demonstrates that Shredder falls short in safeguarding facial images from recovery. Through experimentation with the noise injection layer proposed by Shredder on a shallow layer, we observed significant accuracy drops with combined multiplicative and additive noise. On the other hand, simple additive noise with about a 3% drop in accuracy failed to protect images from recovery.

The failure of Shredder [6] could be attributed to two factors: the client's computational limit and the server's knowledge of the partial weights. To ensure the best defense, Shredder requires the noise injection layer to be very deep, but that results in the client taking heavy computation work and is less practical. Meanwhile, a shallow noise injection layer leaves most of the weights to the server. With rich information about public model weights available, the server can learn a strong and accurate representation of the private network held by the client, which leads to a successful MIA. Thus, the key challenge behind a practical CI framework is to defend against an adversarial server's reconstruction ability when the client only holds a minimal portion of the network.

To address the current deficiencies in privacy-preserving edge-cloud collaborative inference scenarios, in this work,

we propose a novel mechanism of "selective ensemble." Our intuition is as follows. Since the reconstruction ability of the adversarial server is powerful, it may be wiser to hide the client's private model than to defend it directly against the model reconstruction attack. We propose to leverage model ensemble on the cloud and secret private masking on the user side to achieve a "Schrödinger's model," where a reconstruction attack can lead to a successful, but different, shadow network than the client's true network.

Based on the above analysis, we propose *Ensembler*, a novel and secure CI framework designed to substantially increase the adversary's effort required to recover the client's input. The proposed architecture consists of multiple parallel networks, but only a subset of them is actually activated by the client during inference. An existing MIA on the network will lead to a seemingly successful reconstruction, but such reconstruction does not represent the client's real network weights. We make the following contributions in this paper:

- We identify and address the key challenge in practical private collaborative inference, i.e., when the model split location between the client and the server is shallow, a direct defense fails to work.
- We reveal that a secret selection of a subset of the networks in an ensemble architecture can be useful for defending against model inversion attacks.
- We propose *Ensembler*, a standalone framework that can not only significantly increase the adversary server's reconstruction difficulty but also be seamlessly integrated with existing complex algorithms to construct practical and private inference architectures.

To verify the effectiveness of *Ensembler*, we conduct experiments on multiple datasets. Compared to models without using model ensemble, when defending against MIA, *Ensembler* achieves up to 43.5% decrease in structural similarity and 40.5% decrease in peak signal noise ratio. In addition, compared to the previous popular approach, Shredder [6], *Ensembler* better protects the client's sensitive information with a 10.7% decrease in peak signal noise ratio, highlighting its effectiveness even as a standalone framework.

II. Background

A. Collaborative Machine Learning

The development of mobile graphic processing units (GPUs) has ushered in a new era where machine learning tasks are increasingly deployed with a portion of the computation being handled by edge devices. Related areas include federated learning, where multiple edge devices jointly train a DL model [7, 8]; split learning, where a DL model is split into two or more parts, and the client and server jointly train it [9]; and collaborative inference, where a DL model is split, with only a portion deployed on the server to provide services [4, 10]. In this paper, we will focus on the inference part and assume that the training phase of DL models is secure. Though the training phase is sometimes also susceptible to adversarial attacks aimed at stealing sensitive information [11, 12], private inference is still more prevalent in most practical scenarios.

There have been multiple works addressing this formidable challenge of safeguarding the client's sensitive information in collaborative inference scenarios. [13, 14] provides an extensive discussion on different algorithmic and architectural choices and their impacts on privacy protection. For simplicity and without loss of generalization, we will group existing approaches into two categories in this paper: encryption-based algorithms that guarantee privacy at the cost of thousands of times of time efficiency [15, 16, 17, 18], and perturbation-based algorithms that operate on the intermediate layers of a DL architecture, introducing noise to thwart the adversary's ability to recover client input [6, 10, 5, 19]. Since perturbation-based algorithms directly operate on the intermediate outputs from the client, they incur minimal additional complexity during the inference process. As opposed to guaranteed privacy provided by encryption-based algorithms, perturbation-based algorithms suffer from the possibility of privacy leakage, meaning sensitive private information may still be recoverable by the adversarial server despite the introduced perturbations, but they are nonetheless more practical to use due to the less additional costs associated with the methods.

B. Threat model

In this paper, we consider the collaborative inference task between the client and the server, which acts as a semi-honest adversarial attacker that aims to steal the raw input from the client. Formally, we define the system as a collaborative inference on a pre-trained DNN model, $\mathbf{M}(x, \theta)$, where the client holds the first and the last few layers (i.e. the "head" and "tail" of a neural network), denoted as $\mathbf{M}_{c,h}(x, \theta_{c,h})$ and $\mathbf{M}_{c,t}(x, \theta_{c,t})$. The rest of the layers of DNN are deployed on the server, denoted as $\mathbf{M}_s(x, \theta_s)$. θ is the trained weights of M, where $\theta = \{\theta_{c,h}, \theta_s, \theta_{c,t}\}$. The complete collaborative pipeline is thus to make a prediction of incoming image x with $\mathbf{M}_{c,t}[\mathbf{M}_s[\mathbf{M}_{c,h}(x)]]$.

During the process, the server has access to θ_s and the intermediate output $\mathbf{M}_{c,h}(x)$. In addition, we assume that it has a good estimate of the DNN used for inference. That is, it has auxiliary information on the architecture of the entire DNN, as well as a dataset in the same distribution as the private training dataset used to train the DNN. However, it does not necessarily know the hyper-parameters, as well as the engineering tricks used to train the model. Since the server is a computation-as-a-service (CaaS) provider, it is assumed to have reasonably large computation resources. While it is powerful in computing, the server is restricted from querying the client to receive a direct relationship between raw input x and intermediate output $\mathbf{M}_{c,h}(x)$.

To reconstruct raw input x from the intermediate output $\mathbf{M}_{c,h}(x)$, the server adopts a common model inversion attack [4, 5, 20]. To obtain x's reconstruction, MIA aims to obtain the inverse of $\mathbf{M}_{c,h}$, which will be denoted as $\mathbf{M}_{c,h}^{-1}$. The server constructs a shadow network $\tilde{\mathbf{M}}(x, \tilde{\theta_{c,h}}, \theta_s, \tilde{\theta_{c,t}})$: $\{\tilde{\mathbf{M}_{\mathbf{c,h}}}, \mathbf{M}_s, \tilde{\mathbf{M}_{\mathbf{c,t}}}\}$ such that $\tilde{\mathbf{M}}$ simulates the behavior of $\mathbf{M}$. After training $\tilde{\mathbf{M}}$, the adversarial server is able to obtain a representation $\tilde{\mathbf{M}_{\mathbf{c,h}}}$ such that $\tilde{\mathbf{M}_{\mathbf{c,h}}}(x) \sim \mathbf{M}_{c,h}(x)$. As the

next step, with the help of a decoder, $\tilde{\mathbf{M}}_{\mathbf{c,h}}^{-\mathbf{1}} \sim \mathbf{M}_{c,h}^{-1}$, to reconstruct the raw image from intermediate representation, it is able to reconstruct the raw input from $\mathbf{M}_{c,h}(x)$.

C. Assumptions of other related works

In this section, we provide an overview of various attack models and the assumptions adopted in other works related to collaborative inference (CI) under privacy-preserving machine learning (PPML). Since different works potentially use different collaboration strategies between the client and the server, we will use the generic notation, where $\mathbf{M}_c$ is held by the client, and $\mathbf{M}_s$ is held by the server. Generally, the attacks from the server will fall into three categories:

Membership Inference Attacks that try to predict if certain attributes, including but not limited to individual samples, distributions, or certain properties, are a member of the private training set used to train $\mathbf{M}(x, \theta)$. If successful, the privacy of the training dataset will be compromised. We refer readers to the survey by [21] and [22] for more details.

Model Inversion Attacks that try to recover a particular input during inference when its raw form is not shared by the client. For example, in an image classification task, the client may want to split $\mathbf{M}$ such that it only shares latent features computed locally to the server. However, upon successful model inversion attacks, the server will be able to generate the raw image for classification tasks based on the latent features. It is important to note that, in this paper, we adopt the same definition of model inversion attacks as of [4]. This term also refers to attacks that reconstruct the private training dataset in other works. We will focus on reconstructing private raw input for the rest of this paper.

Model Extraction Attacks that try to steal the parameters and even hyper-parameters of $\mathbf{M}$. This type of attack compromises the intellectual property of the private model and is often employed as sub-routines for model inversion attacks when the server lacks direct access to the private model's parameters.

Different works also make different assumptions about the capability of the server. First, it is widely accepted that the server has sufficiently yet reasonably large computing power and resources, as its role is often providing ML service. Regarding the auxiliary information on $\mathbf{M}$, they generally fall into three levels:

White Box assumes that the server has full access to architecture details of $\mathbf{M}$ such as the structure and parameters [23]. Different definitions also add different auxiliary available information such as training dataset [23], corrupted raw input [24], or a different dataset [25]. This setting is often associated with attacks that try to reconstruct private training datasets [25, 24, 26].

Black Box assumes that server does not have any information on both $\mathbf{M}$ and the training dataset. However, it is allowed to send unlimited queries to the client to get $\mathbf{M}_c(x)$ [27, 28].

Query-Free restricts the server from querying $\mathbf{M}_c$. While such an assumption greatly limits the reconstruction ability of the adversarial party, there are no limitations on auxiliary information available to the server besides the actual weights of $\mathbf{M}_c$. [4, 29] have both shown that $\mathbf{M}_c$ is still vulnerable to leaking private information of its input when the server has information on the model architecture and training dataset. Our work uses this setting.

III. *Ensembler* ARCHITECTURE

While it is possible to protect sensitive data by increasing the depth of the client network, such depth is often impractical for edge devices due to the computational demands involved. In this section, we present *Ensembler*, a framework that augments the privacy of intermediate information sent by the client without requiring extra computation efforts of the client during inference. *Ensembler* is highly extensible, and it is compatible with existing works that apply noises and perturbation during both DNN training and inference. We will go over the inference pipeline of *Ensembler* in Section III-A, the training stage of this new framework in Section III-C, an intuitive explanation of why *Ensembler* works in Section III-B, and finally the time complexity analysis in Section III-D.

A. Inference Pipeline

As illustrated in Figure 2, *Ensembler* leverages model ensemble on the server to generate a regularized secret $\mathbf{M}_{c,h}$ that is hard to be reconstructed by the adversarial server. It comprises three parts: standard client layers, N different server nets, and a Selector. During the CI pipeline, the client computes the addition of $\mathbf{M}_{c,h}(x)$ and a predefined noise, and transmits the intermediate output to the server. The server then feeds the intermediate output through each of the $\mathbf{M}_s^i$ and reports the output of each $\mathbf{M}_s^i$ to the client. The client then employs a user-defined private selector to perform a selection of the feedback from the server, which activates the results of P out of N nets and combines them. As a final step, it computes the last few layers ($\mathbf{M}_{c,t}(x)$) to classify the input image. We will introduce these separately in this section.

1) Client Layers: During collaborative inference, the client runs a part of the DNN. Under the proposed framework, the client is responsible for running the first h layers $\mathbf{M}_{c,h}$ and the last t layers $\mathbf{M}_{c,t}$. These layers are the same as the client part of a typical collaborative inference framework. $\mathbf{M}_{c,h}$ takes the raw input (often an image) and outputs the intermediate result, whereas $\mathbf{M}_{c,t}$ takes the output from the server as input and outputs the likelihood of each class.

2) Server Nets: On the server side, the network consists of N copies of DNN, with each $M_s^i \in M_s$ corresponding to what the server would normally process in a typical CI pipeline. That is, each $M^i : \{M_{c,h}, M_s^i, M_{c,t}\}$ is a standard pipeline for the inference task. Upon receiving $\mathbf{M}_{c,h}(x) + N(0, \sigma)$, which is the intermediate features generated by the client, the server shall feed this input into each of M_s^i. It outputs N representations of hidden features used for classification and sends all of them back to the client. We denote those outputted feature maps as $M_s(x') = \{M_s^i(x')\}$.

3) Selector and $\mathbf{M}_{c,t}$: To increase the difficulty of the server reconstructing the model and recovering the raw input, a user-defined private selector is applied to the outputs returned

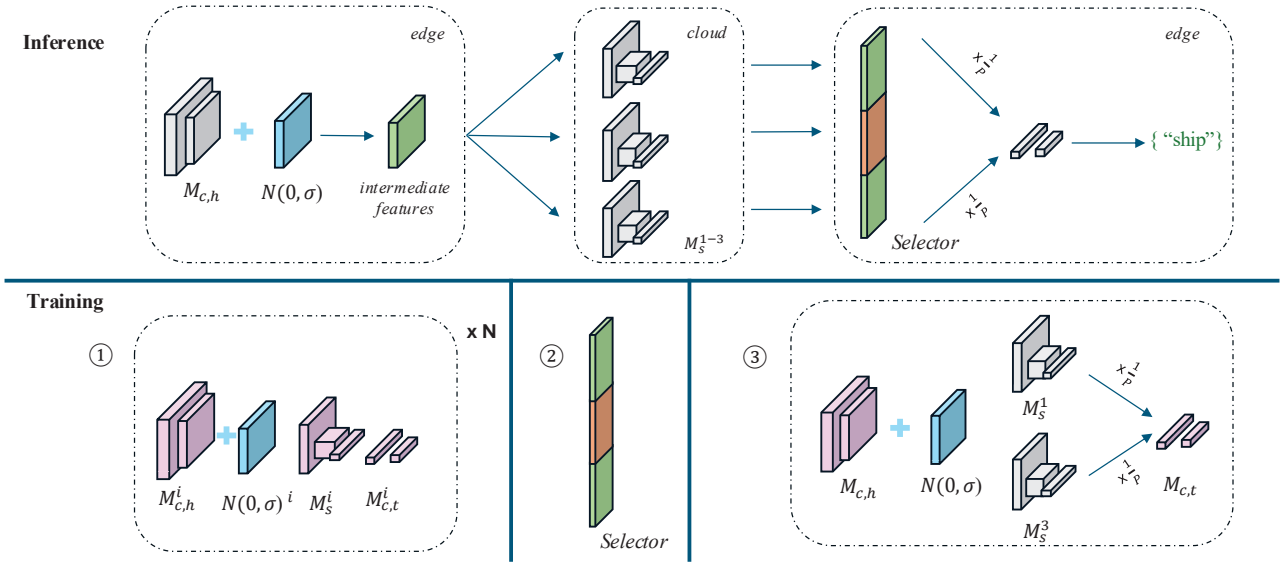

Fig. 2. Illustration of the proposed architecture, *Ensembler*, during inference and training. Unlike traditional CI pipelines, during inference, *Ensembler* deploys N neural networks on the server and uses a private selector to activate P of the N nets, making the reconstruction of the client network infeasible. During training, *Ensembler* follows a three-stage training procedure, and the purple elements are trainable parameters of each stage.

by the server before the client runs $\mathbf{M}_{c,t}$. The selector is a secret activation function, activating a subset of $M_s(x')$. We denote this subset of activated feature maps as $M_s(x')_p$, and there are in total P activated feature maps. Then, the Selector is defined by Eq. 1, where S_i is the activation from the Selector, and $\odot$ is the element-wise multiplication. For simplicity, we consider $S_i = 1/P$. In other words, the client will normalize P of the N feature maps returned by the adversarial server and concatenate them as the input features to $\mathbf{M}_{c,t}$ while keeping those selections private to the server.

$$Sel[M_s(x)] = Concat[S_i \odot f \quad \forall f \in M_s(x')_p] \tag{1}$$

B. Intuition behind Ensembler

In this section, we provide an intuitive explanation of *why* the proposed architecture can defend the client's private information against MIA through a simulated MIA performed by the adversarial server. As defined by the threat model in Section II-B, the server has access to the training dataset, model architecture, and M_s. To perform a MIA, it will construct $\tilde{\mathbf{M}}_{\mathbf{c,h}}$ and $\tilde{\mathbf{M}}_{\mathbf{c,t}}$. Now, let's further assume that each of $M_s \in \{M_s^i\}$ has different weights (how to achieve this will be shown in the next subsection). With brute-forcing all combinations being exponential and unpractical, and since the adversarial server does not know the user-defined selector, there remain two possible MIA constructions. The server could either construct the shadow network with a random subset of $\{M_s^i\}$, or it could use the entire $\{M_s^i\}$. We will show that both methods result in $\tilde{\mathbf{M}}_{\mathbf{c,h}}(x) \not\sim \mathbf{M}_{c,h}(x)$. Without loss of generality, for the first approach, the server is assumed to construct $\tilde{\mathbf{M}}_{\mathbf{c,h}}$ from a single M_s^i that is a subset of the networks selected by the Selector.

Proposition 1: Reconstructing input from single neural network $\mathbf{M}_s^i$ results in $\tilde{\mathbf{M}}_{c,h}(x) \not\sim \mathbf{M}_{c,h}(x)$.

Let's consider the most extreme case, where the Selector selects $\mathbf{M}_s^1$ and $\mathbf{M}_s^2$, and the adversarial server is constructing its shadow network with $\mathbf{M}_s^1$. Since $\mathbf{M}_{c,h}$ is trained with both networks, the gradient of weights of $\mathbf{M}_{c,h}$ is calculated as (roughly) $\eta(\nabla_x \mathbf{M}_s^1 + \nabla_x \mathbf{M}_s^2)$ during backpropogation. After minimization, as long as $\nabla_x \mathbf{M}_s^2$ is not zero, the weights of $\mathbf{M}_{c,h}$ will be affected by both $\mathbf{M}_s^1$ and $\mathbf{M}_s^2$ (we will show how to let the gradient not be zero through loss regularization in the next section). Since $\mathbf{M}_s^1$ and $\mathbf{M}_s^2$ have different weights, their gradient will point to different directions. A shadow network $\tilde{\mathbf{M}}_{c,h}$ trained on only $\mathbf{M}_s^1$ will represent different weights compared to $\mathbf{M}_{c,h}$, and thus the decoder based on $\tilde{\mathbf{M}}_{c,h}$ will also reconstruct incorrect client's secret input.

Proposition 2: Reconstructing input from the entire set $\{\mathbf{M}_s\}$ results in $\tilde{\mathbf{M}}_{c,h}(x) \not\sim \mathbf{M}_{c,h}(x)$.

A similar conclusion from the analysis in the first case may be derived. Let's still use the simple example of $\mathbf{M}_s^1$ and $\mathbf{M}_s^2$. This time, the Selector selects only $\mathbf{M}_s^1$, but the server decides to train on both $\mathbf{M}_s^1$ and $\mathbf{M}_s^2$. It is not hard to see that the weights of $\mathbf{M}_{c,h}$ and $\tilde{\mathbf{M}}_{c,h}$ are just swapped with each other, and now, the decoder is still based on incorrect weights and still leads to an incorrect reconstruction of $\mathbf{M}_{c,h}(x)$.

C. Training Stages

As mentioned above, the key idea behind *Ensembler* is to use N **different** networks with a private Selector to obfuscate MIA. To achieve the N different networks and make sure $M_{c,h}$ is properly trained with all of the selected subsets, the proposed architecture uses a three-staged training pipeline, as shown in the bottom half of Fig. III.

Stage 1: First, *Ensembler* obtains N different networks by injecting N different noises after each of the network's $M_{c,h}^i$ and training each of the networks. Since randomly initialized noises are quasi-orthogonal to each other and each network

needs to adjust its weights to reverse the effects of the added noise on the intermediate feature map, the resulting weights of $M^i_{c,h}$ will be different from each other. Formally, the first stage optimizes Eq. 2, where $N(0,\sigma)^i$ is a fixed Gaussian noise added to the intermediate output. We choose Gaussian noise because of simplicity in implementation, and we argue that any method leading to distinctive $M_{c,h}$ will be sufficient.

$$L^i_\theta = -\sum_j y_j * log\ M^i_{c,t}(M^i_s[M^i_{c,h}(x) + N(0,\sigma)^i])_j \quad (2)$$

Stage 2: After the first training stage, N different DNNs are obtained. The client secretly selects P out of the N networks as the activation of the Selector.

Stage 3: With the Selector defined, the client is now able to aggregate P networks together to form the "ensembled" inference pipeline. At the last stage of the training process, P server networks together form the body of the final network, and their weights are frozen. The client re-trains its private network $M_{c,h}$ and $M_{c,t}$, with a newly defined Gaussian noise added to the output features of $M_{c,h}$. This Gaussian noise follows the same definition as the Gaussian noise in Stage 1's training. Formally, this stage's training loss follows Eq. 3 by adding a high penalty to the model when the gradient descends only to the direction of some single server net M^i_s. It is worth noting that this loss has two terms. The first part is a standard cross-entropy loss used to train a classification network, and the second part is a regularization term to enforce M to learn a joint $M_{c,h}$ and $M_{c,t}$ representation from all of the P server networks. In the equation, CS is the cosine similarity, and λ is a hyperparameter to control the regularization strength. The idea behind such a regularization is that if $M_{c,h}$ only learns from one single network, then its weights will be very similar to one of the previous $M^i_{c,h}$. Thus, we regularize it to be as quasi-orthogonal to all of the previous $M^i_{c,h}$.

$$\begin{aligned} L_\theta &= \sum_i^{i\in P} CE + Regularization, \quad where \\ CE &= -\sum_j [y_j * log M_{c,t}(Sel[M^i_s[M_{c,h}(x) + N(0,\sigma)]])_j] \\ Regularization &= \lambda \max_i [CS(M_{c,h}(x), M^i_{c,h}(x))] \end{aligned} \quad (3)$$

D. Time Complexity of Ensembler

From the previous subsections, it is clear that the inference time complexity of the proposed framework on the client is the same as the standard non-private inference pipeline. On the server, it is N times the individual network on a single-core GPU, and there is negligible extra communication cost between the client and the server. However, it is noteworthy to emphasize that since each M^i_s is independent of each other, the proposed framework is friendly to parallel execution and even multiparty (multi-server) inference. Under those settings, the theoretical time complexity of $O(N)$ would be replaced with lower practical time costs.

Regarding the MIA's time complexity, following the analysis in Section III-B, it is not hard to see that the expected MIA time of a single server is $O(2^N)$. Since an arbitrary reconstruction on some subset of the network will lead to a reasonable shadow network $\tilde{\mathbf{M}}_{\mathbf{c,h}}$, the server has no way of telling whether its reconstruction is an actual representation of the client's network. Thus, to ensure optimal reconstruction, it must use brute force and try all possibilities. In addition, under multiparty inference scenarios, without access to a subset of the activated networks, the adversarial server will not be able to perform the desired reconstruction.

IV. Experiments and Evaluations

A. Implementation details

During the experiment, we consider the most strict setting, where h=1 and t=1 on a ResNet-18 [31] architecture for three image classification tasks, CIFAR-10 [32], CIFAR-100 [32], and a subset of CelebA-HQ [33]. That is, the client only holds the first convolutional layer as well as the last fully connected layer, which is also the minimum requirement for our framework. For CIFAR-10, the intermediate output's feature size is [64x16x16], for CIFAR-100, we remove the MaxPooling layer and the intermediate output's feature size is [64x32x32], and for CelebA, the intermediate output's feature size is [64x64x64]. We consider the ensembled network to contain 10 neural networks (N=10), each being a ResNet-18. The selector secretly selects $\{4,3,5\}$ out of the 10 nets (P=$\{4,3,5\}$), respectively. The adversarial server is aware of the architecture and the training dataset. It constructs a shadow network $\tilde{\mathbf{M}}_{c,h}$ consisting of three convolutional layers with 64 channels each, with the first one simulating the unknown $\mathbf{M}_{c,h}$, and the other two simulating the Gaussian noise added to the intermediate output. It also has $\tilde{\mathbf{M}}_{c,t}$ with the same shape as $\mathbf{M}_{c,t}$. An adaptive shadow network learns from all 10 server nets with an additional activation layer that is identical to the selector. For any noises added to the intermediate outputs during the training and inference stage, we consider a fixed Gaussian noise $\mathbf{g} \sim N(0, 0.1)$.

B. Experiment Setup

We employ two key metrics: Structural Similarity (SSIM) and Peak Signal Noise Ratio (PSNR). Higher SSIM and PSNR values indicate better reconstruction quality and thus worse defense. In addition, we report the changes in classification accuracy (ΔAcc) after adding the defense mechanism. As our proposed architecture operates in parallel with existing perturbation methods, we mainly compare our method with its single network counterpart (Single) [30], where only one $M^i : \{M_{c,h}, M^i_s, M_{c,t}\}$ is used. In the ablation study, we provide a deeper analysis using CIFAR-10, and we compare the following methods against *Ensembler*: no protection (None), training the noise injected to Single (Shredder [6]), adding dropout layer in the single network or ensembled network, but without the first stage training (DR-single and DR-10) [34]. For the proposed architecture, we evaluate the performance of both reconstruction attacks discussed in Section III-B. For reconstruction with a single server net, we report the best reconstruction, or worst defense, in terms of SSIM (Ours -

TABLE I

DEFENSE QUALITY OF *Ensembler* AND SINGLE [30] ACROSS DATASETS. FOR SSIM AND PSNR, LOWER VALUES INDICATE BETTER DEFENSE QUALITY. WE REPORT ON THE TWO RECONSTRUCTION ATTACKS DISCUSSED IN SECTION III-B. FOR RECONSTRUCTING WITH A SINGLE NETWORK, WE REPORT THE STRONGEST RECONSTRUCTION ATTACK (LEAST DEFENSE QUALITY) OUT OF N NETWORKS.

	CIFAR-10			CIFAR-100			CelebA-HQ		
Name	ΔAcc	SSIM ↓	PSNR ↓	ΔAcc	SSIM ↓	PSNR ↓	ΔAcc	SSIM ↓	PSNR ↓
Single	2.15%	0.39	7.53	-0.97%	0.46	8.52	-1.24%	0.27	14.31
Ours - Adaptive	-2.13%	0.06	5.98	0.31%	0.09	4.77	2.39%	0.09	13.37
Ours - SSIM	-2.13%	0.29	4.87	0.31%	0.26	5.07	2.39%	0.18	12.06
Ours - PSNR	-2.13%	0.22	5.53	0.31%	0.26	5.07	2.39%	0.18	12.06

TABLE II

DIFFERENT DEFENSE MECHANISMS WITH CIFAR-10. THE LAST THREE ARE THE PROPOSED FRAMEWORK. THE UNDERSCORED NUMBER REPRESENTS THE BEST DEFENSE FROM OTHER EXISTING METHODS.

Name	Δ Acc	SSIM ↓	PSNR ↓
None	0.00%	0.49	9.86
Shredder	-2.92%	<u>0.29</u>	6.70
Single	2.15%	0.39	7.53
DR-single	2.70%	0.35	<u>6.67</u>
DR-10 - SSIM	1.42%	0.37	7.35
DR-10 - PSNR	1.42%	0.32	7.96
Ours - Adaptive	-2.13%	0.06	5.98
Ours - SSIM	-2.13%	0.29	4.87
Ours - PSNR	-2.13%	0.22	5.53

SSIM) and PSNR (Ours - PSNR) among the N networks. For the reconstruction using all networks, we denote it as (Ours - Adaptive). We run the experiments on an A6000 GPU server using Python and PyTorch. For Section 4, we used a mixture of A6000 and T4 GPU. In addition, we report the real-world time delay of the proposed method compared with its single counterpart and a recent encryption-based method [35]. We evaluate the inference time on a Raspberry Pi communicating with the A6000 server over a wired network.

C. Comparison of Results

We provide the quantitative evaluations of *Ensembler* compared to its non-ensembled counterpart (Single [30]) in Table I. It could be seen that the proposed framework significantly decreases the reconstruction quality of the adversarial party. *Ensembler* incurs a 2.13% drop in classification accuracy compared to the model without any protection, which is acceptable compared to its advantage in protecting the privacy of the client's raw input. It should be noted that the adaptive MIA strategy performs worse than the best reconstruction with a single network, which is expected. While Equation 3 constraints $M_{c,h}$ to learn from all server networks, it might still have one that is more "favored". Thus, a reconstruction based on that favored network yields the best results. On the other hand, a reconstruction based on all networks might not select the same one as its "favored" network, which causes the shadow network's weights to significantly deviate from $M_{c,h}$.

Moreover, we delve into a more thorough analysis of *Ensembler*'s performance on the CIFAR-10 dataset compared to other existing methods. The results are summarized in Table II. It could be observed that *Ensembler* defends the client's input on par with Shredder [6] on structural similarity, and exceeds it on peak signal-noise ratio. In addition, it should be noted that Shredder [6] and dropout defense [34] can be combined with *Ensembler* together. The additive noise $N(0, \sigma)$ in the third stage could be replaced by Shredder's trained noise, and dropout can also be added to the network's FC layer to perform further protection of the client's sensitive data.

TABLE III

TIME TAKEN (SECONDS) TO RUN A BATCH OF 128 IMAGES THROUGH DIFFERENT DEFENSE STRATEGIES.

Name	Client	Server	Communication	Total
Standard CI	0.66	0.98	2.30	3.94
Ensembler	0.66	1.02	2.45	4.13
STAMP [35]	–	–	–	309.7

D. Latency

Time required by the standard CI setting and *Ensembler* is provided in Table III. In addition, STAMP [35] is a hardware-optimized version of encryption-based private inference, and we show their reported results (LAN-GPU). For all experiments, we use the setting in STAMP to run ResNet-18 on a batch size of 128 images. *Ensembler* incurs only **0.19 seconds (4.8%)** overhead compared with running the standard CI pipeline, but with much stronger protection on the client's sensitive data. It can also be observed that the increment of time comes from two parts, with more server computing taking the minor role and more communication between the client and the server taking the major role. In the future, more attention on improving client-server communication is pivotal for a more efficient and practical collaborative inference.

V. CONCLUSION

During collaborative inference, the client's sensitive information is prone to model inversion attacks from the adversarial server, especially when the client only holds a very small portion of the network. In this paper, we present a novel collaborative inference framework, *Ensembler*, designed to significantly increase the complexity of reconstruction for adversarial parties. *Ensembler* leverages a selective model ensemble on the adversarial server and a secret Selector to defend against the server's model inversion attacks. In addition, it seamlessly aligns with existing methods that introduce diverse forms of noise to intermediate outputs, potentially yielding robust and adaptable architectures if combined with them. Lastly, *Ensembler* is a latency-friendly framework that incurs only marginal overhead during inference. Our experiments highlight the substantial deterioration in reconstruction quality for images safeguarded by *Ensembler* when compared to those without its protection.

REFERENCES

[1] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.

[2] X. Hu, L. Chu, J. Pei, W. Liu, and J. Bian, "Model complexity of deep learning: A survey," *Knowledge and Information Systems*, vol. 63, pp. 2585–2619, 2021.

[3] R. Qin, D. Liu, C. Xu, Z. Yan, Z. Tan, Z. Jia, A. Nassereldine, J. Li, M. Jiang, A. Abbasi *et al.*, "Empirical guidelines for deploying llms onto resource-constrained edge devices," *arXiv preprint arXiv:2406.03777*, 2024.

[4] Z. He, T. Zhang, and R. B. Lee, "Model inversion attacks against collaborative inference," in *Proceedings of the 35th Annual Computer Security Applications Conference*, 2019, pp. 148–162.

[5] R. Lu, S. Shi, D. Wang, C. Hu, and B. Zhang, "Preva: Protecting inference privacy through policy-based video-frame transformation," in *2022 IEEE/ACM 7th Symposium on Edge Computing (SEC)*, 2022, pp. 175–188.

[6] F. Mireshghallah, M. Taram, P. Ramrakhyani, A. Jalali, D. Tullsen, and H. Esmaeilzadeh, "Shredder: Learning noise distributions to protect inference privacy," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 3–18.

[7] X. Yu, L. Cherkasova, H. Vardhan, Q. Zhao, E. Ekaireb, X. Zhang, A. Mazumdar, and T. Rosing, "Async-hfl: Efficient and robust asynchronous federated learning in hierarchical iot networks," in *Proceedings of the 8th ACM/IEEE Conference on Internet of Things Design and Implementation*, 2023.

[8] D. Yaldiz, T. Zhang, and S. Avestimehr, "Secure federated learning against model poisoning attacks via client filtering," in *ICLR 2023 Workshop on Backdoor Attacks and Defenses in Machine Learning*, 03 2023.

[9] M. G. Poirot, P. Vepakomma, K. Chang, J. Kalpathy-Cramer, R. Gupta, and R. Raskar, "Split learning for collaborative deep learning in healthcare," *arXiv preprint arXiv:1912.12115*, 2019.

[10] S. Osia, A. Taheri, A. Shahin Shamsabadi, K. Katevas, H. Haddadi, and H. Rabiee, "Deep private-feature extraction," *IEEE Transactions on Knowledge and Data Engineering*, 2018.

[11] J. Li, A. S. Rakin, X. Chen, Z. He, D. Fan, and C. Chakrabarti, "Ressfl: A resistance transfer framework for defending model inversion attack in split federated learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 10 194–10 202.

[12] W. Zhang, S. Tople, and O. Ohrimenko, "Leakage of dataset properties in {Multi-Party} machine learning," in *30th USENIX security symposium (USENIX Security 21)*, 2021.

[13] R. Xu, N. Baracaldo, and J. Joshi, "Privacy-preserving machine learning: Methods, challenges and directions," *arXiv preprint arXiv:2108.04417*, 2021.

[14] C. Xu, F. Yu, Z. Xu, N. Inkawhich, and X. Chen, "Out-of-distribution detection via deep multi-comprehension ensemble," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 21–27 Jul 2024, pp. 55 465–55 489.

[15] W. Z. Srinivasan, P. Akshayaram, and P. R. Ada, "Delphi: A cryptographic inference service for neural networks," in *Proc. 29th USENIX secur. symp*, vol. 3, 2019.

[16] D. Rathee, M. Rathee, N. Kumar, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma, "Cryptflow2: Practical 2-party secure inference," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 325–342.

[17] K.-Y. Lam, X. Lu, L. Zhang, X. Wang, H. Wang, and S. Q. Goh, "Efficient fhe-based privacy-enhanced neural network for trustworthy ai-as-a-service," *IEEE Transactions on Dependable and Secure Computing*, 2024.

[18] J. Li and J. Xiong, "xmlp: Revolutionizing private inference with exclusive square activation," *arXiv preprint arXiv:2403.08024*, 2024.

[19] W. Sirichotedumrong and H. Kiya, "A gan-based image transformation scheme for privacy-preserving deep neural networks," in *2020 28th European Signal Processing Conference (EUSIPCO)*, 2021, pp. 745–749.

[20] A. Dosovitskiy and T. Brox, "Inverting visual representations with convolutional networks," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4829–4837.

[21] H. Hu, Z. Salcic, L. Sun, G. Dobbie, P. S. Yu, and X. Zhang, "Membership inference attacks on machine learning: A survey," *ACM Computing Surveys (CSUR)*, vol. 54, 2022.

[22] A. Salem, G. Cherubin, D. Evans, B. Köpf, A. Paverd, A. Suri, S. Tople, and S. Zanella-Béguelin, "Sok: Let the privacy games begin! a unified treatment of data inference privacy in machine learning," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 327–345.

[23] X. Liu, L. Xie, Y. Wang, J. Zou, J. Xiong, Z. Ying, and A. V. Vasilakos, "Privacy and security issues in deep learning: A survey," *IEEE Access*, vol. 9, pp. 4566–4593, 2021.

[24] Y. Zhang, R. Jia, H. Pei, W. Wang, B. Li, and D. Song, "The secret revealer: Generative model-inversion attacks against deep neural networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 253–261.

[25] Q. Wang and D. Kurz, "Reconstructing training data from diverse ml models by ensemble inversion," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 2909–2917.

[26] N. Haim, G. Vardi, G. Yehudai, O. Shamir, and M. Irani, "Reconstructing training data from trained neural networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 22 911–22 924, 2022.

[27] Y. Xu, X. Liu, T. Hu, B. Xin, and R. Yang, "Sparse black-box inversion attack with limited information," in *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.

[28] M. Kahla, S. Chen, H. A. Just, and R. Jia, "Label-only model inversion attacks via boundary repulsion," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 15 045–15 053.

[29] S. Ding, L. Zhang, M. Pan, and X. Yuan, "Patrol: Privacy-oriented pruning for collaborative inference against model inversion attacks," 2023.

[30] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography: Third Theory of Cryptography Conference, TCC 2006, New York, NY, USA, March 4-7, 2006. Proceedings 3*. Springer, 2006, pp. 265–284.

[31] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[32] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[33] H. Zhu, W. Wu, W. Zhu, L. Jiang, S. Tang, L. Zhang, Z. Liu, and C. C. Loy, "CelebV-HQ: A large-scale video facial attributes dataset," in *ECCV*, 2022.

[34] Z. He, T. Zhang, and R. B. Lee, "Attacking and protecting data privacy in edge–cloud collaborative inference systems," *IEEE Internet of Things Journal*, vol. 8, no. 12, pp. 9706–9716, 2021.

[35] P. Huang, T. Hoang, Y. Li, E. Shi, and G. E. Suh, "Efficient privacy-preserving machine learning with lightweight trusted hardware," *arXiv preprint arXiv:2210.10133*, 2022.